

2026-06-02 회의록(수업 진행 기록)

1) 기본 정보

- **일사:** 2026-06-02
- **형태:** 대면 + 비대면 혼합 수업(Zoom/온라인 병행 추정)
- **참석:** 강사, 수강생(대면/비대면). 일부 수강생 개별 질의(예: “승현” 언급)
- **비고(확인 필요):** 기관/과정명, 정확한 수업 시간, 출석 인원

2) 진행 요약

2.1 수업 운영/공지

- 수업 중 **마이크/배터리 문제**로 일시적 진행 불편 발생(휴대폰 배터리 부족, 연결 끊김 등).
- 코드 전달 방식 변경:
 - 채팅으로 코드 제공 시 **들여쓰기/정렬 문제**가 반복되어,
 - 이후 **Notion(“로션/노션” 으로 언급)**에 코드 업로드 후 복사-붙여넣기 방식으로 전환.
- 수강 환경 권장:
 - 비대면 수강생에게 **듀얼 모니터 투자 권장**(학습 효율/생산성 관점).

2.2 평가/테스트 공지

- **금요일 오전:** 셀프 테스트(시험) 진행 예고
- 형태: **빈칸 채우기(약 10개 내외) 중심**, VS Code에서 실행하는 방식
- 생성형 AI로 인한 **컨닝 가능성** 언급(대응 고민 중)
- 금요일에는 **생성형 AI 기초 강의**도 짧게 진행 예정(테스트와 분리 운영 언급).

3) 주요 학습 내용(클래스)

3.1 클래스 변수 vs 인스턴스 변수

- **클래스 변수(Class Variable)**
 - 모든 객체가 **같은 값을 공유**.
 - 예시: company_name, total_members(객체 생성 시 카운트 증가)
- **인스턴스 변수(Instance Variable)**
 - 객체(인스턴스)마다 **서로 다른 값을** 가짐.
- 포인트 정리:
 - “공유해야 하는 값” 은 클래스 변수
 - “개별 객체별 값” 은 인스턴스 변수

3.2 객체 생성 시 클래스 변수 업데이트

- 객체 생성(생성자 실행) 시 total_members += 1 형태로 **자동 카운트 증가**하도록 구현.
- 오타/변수명 불일치(total_member vs total_members 등)로 에러 발생 사례 공유 및 **오타 주의** 강조.

3.3 클래스 변수 변경의 전역 영향

- TeamMember.company_name = "..."
- 회사명 변경 시 **모든 인스턴스 출력에 즉시 반영**됨을 실습으로 확인.

4) 매직 메소드(특수 메소드) 소개

4.1 __str__, __repr__ 사용 목적

- **__str__:** print() 또는 str() 호출 시 **사람이 읽기 좋은 형태로** 출력
- **__repr__:** repr() 호출 시 **디버깅/객체 표현**에 유용한 출력 제공
- 메소드 미구현 시:
 - 객체 출력이 **주소값 형태**로만 나타나 이해가 어려움.
- 매직 메소드 종류가 많으며, 필요 시 **공식 문서(데이터 모델)** 참고 안내.

5) 메소드 유형: 인스턴스/클래스/정적 메소드

5.1 클래스 메소드(@classmethod)

- 데코레이터 @classmethod 사용, 첫 인자로 cls를 받음.
- 클래스 변수 접근/반환 등에 활용:
 - 예: 현재까지 생성된 팀원 수 반환 return cls.total_members
- **문자열 입력**을 받아 split()로 분해 후 cls(...)로 객체 생성:
 - 예: "Carol DataAnalyst 4500" → (name, role, salary)로 분리하여 인스턴스 생성
- 결과 확인:
 - 문자열 기반으로도 객체 생성 가능, 총 팀원 수가 증가하는지 확인

5.3 정적 메소드(@staticmethod)

- self/cls 없이 **독립 유틸 함수**처럼 동작.
- 예: 연봉 범위 유효성 검사 is_valid_salary(salary)
- 범위 예시: 1000만 ~ 1억(수업 중 임의 범위로 설명)

6) 상속(Inheritance) 및 오버라이딩(Overriding)

6.1 상속 개념

- 공통 기능을 가진 **부모 클래스(Department)**를 만들고,
- 개발/기획/영업/분석 등 **자식 클래스**가 상속받아 확장.

6.2 오버라이딩

- 공통 메소드(예: team_meeting)를 부서별 문화/방식에 맞게 **재정의**하는 개념 강조.
- 수강 포인트:
 - 문법 자체보다 “**설계(아키텍처/계층 구조)**” 이해가 중요.

6.3 실습 흐름(요약)

- 팀/부서 객체 생성 → 팀원 채용 → 각 팀에 멤버 배정(add_member)
- 팀 현황 출력:
 - 팀 정보 출력(인원/예산 등)
 - 멤버 목록 출력(show_members)
 - 팀 연봉 합계(get_total_salary)

7) 모듈/패키지(다음 주제) 도입 및 실습

7.1 모듈 개념

- 코드가 길어지면 하나의 파일에 몰아넣기 어려움 → **기능별 파일 분리**.
- import로 외부 모듈/파일의 함수 · 변수 사용.

7.2 실습(간단 예제)

- calc.py에 add(a, b) 정의
- main.py에서 import calc 후 calc.add(...) 실행
- 실행 방법:
 - 터미널에서 python main.py 형태로 실행(환경에 따라 uv run python ...도 사용)

7.3 미니 프로젝트 샘플(02 폴더) 안내

- 엔트리 포인트: if __name__ == "__main__":
- **메인 파일에서만 실행 흐름**이 시작되도록 구성
- 단순 출력 버전과 메뉴 반복(while) 기반 버전 비교:
 - 메뉴형은 반복문/입력 처리로 구조가 길어짐
- 결론:
 - 모듈화/패키지화는 실무 코드의 기본이며, **클래스 기반 구성**과 연결됨.

8) 이슈/질의응답(정리)

8.1 반복 발생 이슈

- **들여쓰기(Indentation)** 불일치로 오류 빈발
- 복사-붙여넣기 과정에서 괄호/오타 누락 발생(예: 리스트/인덱싱 관련 괄호 누락)

8.2 Q&A 주요 내용

- 주피터 실행 번호(왼쪽 숫자)는 **실행 카운트**이며 결과 확인에만 활용(큰 의미 없음).
- 팀원 수가 이상하게 증가하는 현상:
 - 셀 단위 재실행/누적 실행으로 카운터가 계속 증가하는 점 안내
 - “전역/지역(상태) 개념 이해” 가 필요하다고 재강조

9) 액션 아이템

9.1 수강생 할 일

- **클래스 파트 복습(필수):** 클래스 변수/인스턴스 변수, 매직 메소드, 클래스/정적 메소드, 상속/오버라이딩
- **금요일 오전 테스트 대비:** 빈칸 채우기 형태로 코드 실행이 가능하도록 연습(VS Code 실행 기준)
- **노션 코드 확인 및 동일 환경 재현:** 들여쓰기/오타 점검 포함
- (권장) 비대면 학습 효율 개선을 위한 **듀얼 모니터 구성 검토**

9.2 강사 후속 조치

- 전체 코드/샘플 코드 **노션에 정리 업로드**
- 금요일 테스트 범위/형식 확정 및 공지(확인 필요)

10) 마무리

- 클래스는 라이브러리/패키지 이해의 기반이며, “직접 작성” 보다도 **구조를 읽고 실행 · 수정할 수 있는 수준**이 1차 목표로 제시됨.
- 질의는 수업 종료 전(약 20분)까지 접수 후 마무리.